

The most important change was separating the runtime from the skills. In the prototype, `main.py` did everything. After the split, each request follows one fixed path (guardrail, route, permission check, execute, audit), and every skill only knows its own job. This made the system much easier to test, because we could test the runtime with fake skills and never start the model. Because each part now has one clear job, we can change or add a skill without touching the rest, the same governance and logging code is reused for every request, and failures show up as clear statuses.

The biggest technical challenge was routing order. The campus skill claims broad keywords like "university", so "Where is Shenzhen University Library?" matches both campus and library. If campus came first, library questions were answered with the wrong knowledge. We fixed it by ordering the skills from most specific to most general (translation, library, campus) and taking the first match.

I would make two improvements. First, replace keyword routing with embedding- or LLM-based routing, because requests without a listed keyword now fall through to unmatched. Second, add skill composition, so skills can chain together, for example knowledge lookup, then summary, then translation.